BLOCKCHAIN DEVELOPMENT(WEB 3) LEARNING AND BUILDING GUIDE (BY "LegendaryCode")

NOTE: MAKE SURE TO SCROLL TO THE VERY LAST PAGE, LINKS AND OTHER VALUABLE EXCLUSIVE PRIVATE DETAILS ABOUT WEB3 DEVELOPEMNT ARE INCLUDED

To Become a Web3 developer It Include The Following Steps:

- Learn core web fundamentals (HTML/CSS/JS) Also know
- And then dive into blockchain basics, smart contracts (Solidity for EVM)
- key libraries (Ethers.js/Web3.js) for frontend/backend interaction
- Tools like Hardhat/Foundry for development, and decentralized storage (IPFS).
- Focus on the foundational layers: blockchain (Ethereum/Solana), smart contracts, decentralized storage, and the frontend/backend tools to connect them.

✓ **Core Foundations (Web2 Skills)**

- Programming: JavaScript (essential), TypeScript (recommended as well when grounded on js)
- Frontend: HTML, CSS, React for UI.
- Backend: Node.js, APIs (REST/gRPC), Databases.

✓ **Blockchain & Smart Contracts**

- Blockchain Concepts: Understand decentralized networks, consensus mechanisms, and wallets (MetaMask).
- Smart Contract Languages: Solidity: For Ethereum Virtual Machine (EVM) chains (Ethereum, Polygon, etc.).
- Development Environments/Frameworks: Hardhat, Foundry, Truffle (for EVM).

✓ **Web3 Libraries & Tools (Interaction Layer)**

- Interaction Libraries: Ethers.js or Web3.js (for JS/TS frontend/backend to talk to the blockchain).
- Decentralized Storage: IPFS (InterPlanetary File System).

Testing/Debugging: Brownie (Python-based).

Layered Approach to Learning

1. Master Web Fundamentals: Get solid on JS/TS and a frontend framework (Reactjs)
2. Learn Blockchain Basics: Understand how blockchains, nodes, and wallets work.
3. Dive into Smart Contracts: Learn Solidity and build simple contracts.
4. Connect with Libraries: Use Ethers.js/Web3.js with React to build DApps.
5. Utilize Frameworks: Use Hardhat/Foundry to compile, test, and deploy.
6. Explore Decentralized Tech: Integrate IPFS for storage

❖ HERE'S A ROADMAP FOR A "FRONTEND" OR "FULL-STACK WEB DEVELOPER" TRANSITIONING TO WEB3 DEVELOPMENT

1.0 Fundamentals of Blockchain and Cryptocurrency .

1.1 Understand blockchain basics.

1.2 Learn about cryptocurrencies and tokens.

1.3 Explore smart contracts.

2. Ethereum Basics

2.1 Study Ethereum's architecture.

2.2 Understand gas fees and transactions.

2.3 Learn about Ethereum accounts and wallets.

3. Solidity Programming

3.1 Learn Solidity syntax and structure.

3.2 Practice writing simple smart contracts.

3.3 Understand contract deployment and interaction.

4. Web3.js / Ethers.js/Viem :

4.1 Master interacting with Ethereum nodes.

4.2 Learn to connect frontend to smart contracts.

4.3 Practice reading from and writing to the blockchain.

5. Decentralized Storage :

5.1 Explore IPFS and Filecoin.

5.2 Learn to store and retrieve data on decentralized networks.

6. Frontend Frameworks for Web3 :

6.1 Learn React.js if not already familiar.

6.2 Explore Web3-specific UI libraries (e.g., web3-react, useDApp).

7. Wallet Integration :

7.1 Implement MetaMask and other wallet connections

7.2 Handle account switching and network changes

8. Decentralized Identity :

8.1 Understand concepts like ENS and DIDs

8.2 Implement authentication using wallets.

9. Layer 2 Solutions:

9.1 Learn about scaling solutions (e.g., Optimism, Arbitrum)

9.2 Understand how to interact with L2 networks.

10. DeFi Protocols :

10.1 Study popular DeFi protocols (e.g., Uniswap, Aave)

10.2 Learn to integrate with DeFi protocols

11. NFTs and Token Standards :

11.1 Understand ERC standards (ERC20, ERC721, ERC1155) :

11.2 Learn to mint and trade NFTs

12. Testing and Security

12.1 Master testing smart contracts

12.2 Learn about common security vulnerabilities and best practices

13. Decentralized Autonomous Organizations (DAOs)

13.1 Understand DAO structures and governance.

13.2 Explore tools for creating and managing DAOs.

❖ HOW TO BECOME A BLCOKCHAIN(WEB 3) DEVELOPER

Prerequisite (i):

- ✓ **Fundamentals of Blockchain**
  1. **Blockchain**
  2. **EVM**
  3. **Cryptp currency**
  4. **Smart contracts**
  5. **Consensus**
  6. **Transaction**
  7. **Gas fee**
  8. **Crypto graphu etc.**

Prerequisites (ii)

- ✓ **If you are not coming from a Web2 Background :**
  1. **Databases**
  2. **How front-end interacts with back-end**
  3. **Basic understanding on how wallet works .**
  4. **Get basic understanding of crypto wallet .**
  5. **Blockchain explorer**

- ✓ **For the back-end Programming Lang:**
  - a. **Solidity (for Ethereum & all Eth layer 2 - EvM) -**
    DOCS:
    https://docs.soliditylang.org/en/v0.8.30/
  - b. **Nodejs(foYou must know Nodejs before you understand reactjs (frontEnd framework)) -**
    DOCS:
    https://nodejs.org/en
  - c. **Rust (for SVM - Solana)**
    DOCS: https://rust-lang.org/

- ✓ **Development FrameWork (in other to write the smart contracts you can use this amazing framework)**
  - a) **Remix (www. remix.ethereum.org) -**
    DOCS:
    https://remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.26+commit.8a97fa7a.js
  - b) **HardHat: it helps you write, test, compile, & deploy our smart contract. -**
    DOCS:
    https://hardhat.org/
  - c) **Foundry :- Foundry is used in blockchain development to write, compile, test, debug, and deploy smart contracts for Ethereum-compatible networks**
    DOCS:
    https://getfoundry.sh/
  - d) **Base Docs: Documentation needed when building on Base EVM layer 2 blockchain. - https://docs.base.org/get-started/base**

Note: important thing to keep in mind when writing smart contract is to make sure to test your smart contract well before testing it on the "main network".
You can try deploying it either on the testnetwork or locally development network

- ✓ **Skills for Front-End Web Development (blockchain dev needed skill)**
  - a) **HTML: Official Documentation resource site for HTML: https://developer.mozilla.org/en-US/docs/Web/HTML**
  - b) **CSS: Official Documentation resource site for CSS: https://developer.mozilla.org/en-US/docs/Web/CSS**
  - c) **JAVASCRIPT: Official Documentation resource site for Javascript: https://www.javascript.com/ https://www.w3schools.com/Js/**

- ✓ **Front - End framework (Needed)**
  - a)
    Reactjs: Official Documentation resource site for Reactjs: https://react.dev/
  - b) **Nextjs: Official Documentation resource site for Nextjs : https://nextjs.org/docs**

- ✓ **JavaScript Libraries(How to make the front-end & back-end interactive), connecting the both**

Choose the best for you :

a) **Ether.js** -
DOCS:
https://docs.ethers.org/v5/

b) **Web3.js**

If your application requires you to read or write from the blockchain to the client , then use among this JS Library. -
DOCS:
https://web3js.readthedocs.io/en/v1.10.0/

Note: In other to interact with the blockchain such as : "signing messages" , "sending transactions" . You will either have to set-up your own wallet such as "MetaMask"

- ✓ **Database (Decentralized database , file storage for decentraluzed systems)**
  These 3 List Below Are Various Choice You Can Make When Choosing :-

a) **IPFS**
DOCS:
https://ipfs.tech/

b) **Filecoin** -
DOCS:
https://filecoin.io/

c) **Arweave**
DOCS:
https://arweave.org/

❖ BUILDING PRODUCTS / ONCHAIN (EVM CHAIN) LINKS (GUIDE) - (IN SUMMARY):

- ✓ **BLOCKCHAIN "FRONTEND DEVELOPMENT" LEARNING & BUILDING RESOURCES**
  1. **React** https://react.dev/
  2. **Nextjs** **https://nextjs.org/docs**
  3. **TwindCSS OR Vanilla CSS** **https://tailwindcss.com/docs/installation/using-vite** / https://developer.mozilla.org/en-US/docs/Web/CSS

- ✓ **BLOCKCHAIN "BACK-END DEVELOPMENT()" LEARNING & BUILDING RESOURCES**

1. **Solidity: Official documentation for , the smart contract programming languages for ethereum (and all layer2 EVM blockchain like "base chain") - https://docs.soliditylang.org/en/v0.8.30/**

2. **Cyfrin Updraft: Start your blockchain dev journey here! It offers easy-to-follow tutorials, from setting up smart contracts to mastering solidity. Perfect for begineers looking for hands-on experience. https://updraft.cyfrin.io/courses/solidity**

3. **CryptoZombies: Learn to code blockchain DApps by building your own games. Interactive tutorials make learning fun. https://cryptozombies.io/**

4. **Alchemy University: Alchemys free courses provide in-depth knowledge on Ethereum and web3 dev tools, making it easy to go from novice to pro. Best for diving into decentralized development with EVM focus. https://www.alchemy.com/university/courses**

5. **Speed Run Ethereum: A series of challenges designed by Austin Griffith to help developers master smart contracts and DApp development with Scaffold-ETH. https://speedrunethereum.com/**

Roadmap for blockchain Development: by

i. https://roadmap.sh/blockchain
ii. https://www.cyfrin.io/blog/how-to-become-a-blockchain-solidity-developer

Learn this by "cyfrin" after learning javascript & solidity, to gain how fulstack development works

https://updraft.cyfrin.io/courses/full-stack-web3-development-crash-course/welcome-to-the-course/intro

❖ FRONT-END DEVELOPMENT ROADMAP (FOR BLOCKCHAIN DEVELOPMENT) - SKILLS NEEDED

Master the skills needed to build stunning and functional Web3 interfaces:

Begineers:

- HTML, CSS & JavaScript fundamentals
- React.js basics and component architecture
- State management with Context API/Redux
- UI libraries and responsive design
- Web3 concepts and blockchain basics

Intermediate:

- Web3.js/Ethers.js integration
- Wallet connections (MetaMask, WalletConnect)

- Smart contract interaction from frontend
- Transaction handling and error management
- Client-side data caching and optimization

Advanced:

- Complex DeFi interfaces and dashboards
- NFT marketplace UIs and interactions
- Cross-chain application development
- Data visualization for blockchain data
- Advanced state management patterns

❖ BACK-END DEVELOPMENT ROADMAP (FOR BLOCKCHAIN DEVELOPMENT) - SKILLS NEEDED

Develop secure and efficient smart contracts and blockchain systems

Begineers:

- **Solidity/Rust programming fundamentals**
- **Smart contract development environments**
- **Basic contract structures and patterns**
- **Testing frameworks introduction**
- **Deployment to testnets**

Intermediate:

- **Advanced smart contract patterns**
- **Security considerations and best practices**
- **Gas optimization techniques**
- **Comprehensive testing and automation**
- **Contract upgradeability patterns**

Advanced:

- Oracle integration and external data
- Layer 2 solutions and scaling techniques

- Cross-chain protocol development
- Advanced governance mechanisms
- Formal verification and auditing

❖ OTHER IMPORTANT INFORMATION

➢ Smart Contract Languages & IDEs: ---

1) **Solidity: The primary language for writing smart contracts on Ethereum and EVM compatible blockchain.**

2) **Rust: Used for high-performance blockchain platforms like Solana.**

➢ Development Frameworks & Toolchains: ---

1) **Hardhat: A flexible , javascript-based framework for Ethereum development , featuring a local Ethereum network and tools for testing and deployment .**

2) **Foundry: A Rust-based toolkit that provides tools for "smart contract development", "testing", "deployment", and "interaction with blockchain".**

3) **Truffle: A reliable suite of tools designed to steamline the smat contract development lifecycle.**

4) **Brownie: A python based framwork with built-in testing and debugging capabilities.**

➢ Essential Libraries & Tools: ---

1) **Web3.js: A "javascript library" for building dApps that can interact with the Ethereum blockchain.**

Web3.js helps you connect the "FrontEnd (eg. React.js)" & the "Smart Contract (e.g Solidity)". Web3.js is just a like a bridge between the 2 technology.

2) **Ethers.js: Another JavaScript library that allows dAppsto interact with Ethereum.**

Also, "Ethers.js" helps you connect the "FrontEnd (eg. React.js)" & the "Smart Contract (e.g Solidity)". Web3.js is just a like a bridge between the 2 technology.

3) **OpenZeppelin : Provides secure , audited smart contract standards and libraries for building on Ethereum.**

4) **Chainlink: A decentralized oracle network , that connects smart contract to real-world data and off-chain system.**

- **Wallats: ---**
  1) **MetaMask: A widenly used browser extension wallet for managing digital assetsand interacting with dApps.**
  2) **Rabby Wallet and Trezor: Other popular wallet options for users.**

- **Other Key Tools:**

  1) **Decentralized Storage: Platforms like "IPFS" (InterPlanetay File System) offer decentralized solutions for storing data.**

  2) **Base Docs: Base Documentations when Building on EVM layer 2 blockchain - https://docs.base.org/get-started/base**

  3) **Block Explorers: Tools like Etherscan provide detailed informationon transactions and smart contracts on the blockchain.**

  4) **Security Tools: A range of platforms for smart contract auditing and fuzzing , such as CodeHawks, Aderyn, and Formal Verification tools.**

- **Key Differences of Crypto On-Rampsm & Crypto Off-Ramp**

Crypto On-Ramp:
A service that allow users to exchange "Fiat currency" to "Crypto Currency".
Purpose, to buy crypto currency.
E.g Buying crypto with a credit card

Crypto Off-Ramp:
A service that allow users to exchange "Crypto Currency" to "Fiat Currency".
Purpose , Selling crypto currency for fiat.
E.g cashing out crypto to a bank account.

Examples of On-ramp services include :
Coinbase , binance exchanges/ platform